

Why SQL Basics Decide the Interview Before It Really Starts

Most people preparing for a SQL interview spend their time rehearsing window functions, recursive CTEs, and multi-table joins. That instinct is backwards. The overwhelming majority of SQL interview failures happen in the first five minutes, on questions about `SELECT` and `WHERE` — the stuff that looks too simple to study.

Here's why that matters: interviewers structure SQL rounds like a funnel. If you stumble on basic filtering, they rarely move on to ask about joins, subqueries, or performance tuning — not because those topics don't matter, but because there's no point probing depth when the foundation already cracked. Nailing the basics isn't busywork. It's the gate that determines whether the rest of the conversation even happens.

This lesson walks through the handful of `SELECT`/`WHERE` traps that quietly sink otherwise strong candidates: the `NULL` comparison trap, `DISTINCT` vs `GROUP BY` confusion, Top-N query patterns, `WHERE` vs `HAVING` mix-ups, and why long `OR` chains are a design smell. None of this is exotic. All of it is testable, memorizable, and — once you see it clearly — impossible to get wrong again.

Problem Overview

Strip away the jargon and every SQL interview basics question is really testing one thing: do you understand what a row filter actually does before, during, and after grouping?

Concretely, you'll typically be asked to:

- Retrieve rows that match a condition (`SELECT ... WHERE`)
- Correctly identify missing data (`NULL` handling)
- Return unique values versus computing per-group aggregates (`DISTINCT` vs `GROUP BY`)
- Find the Nth-highest or Nth-lowest row in a result set (`ORDER BY` + `LIMIT` + `OFFSET`)
- Filter on a computed aggregate rather than a raw column (`HAVING` vs `WHERE`)
- Write a clean multi-value condition instead of a chain of `OR` s (`IN`)

None of these require exotic SQL features. They require precision about *when* SQL evaluates each clause, and that precision is exactly what separates a candidate who "kind of knows SQL" from one an interviewer trusts with production queries.

Example

Imagine a table `employees` :

	id	name	department	salary	email
1	Asha	Sales	72000	asha@co.com	
2	Marco	Sales	65000	NULL	
3	Priya	Engineering	98000	priya@co.com	
4	Diego	Engineering	91000	diego@co.com	
5	Wren	Engineering	91000	NULL	

A few questions this table can generate:

- "Find employees with no email on file." A candidate who writes `WHERE email = NULL` gets **zero rows back**, even though Marco and Wren clearly qualify. That's the NULL trap.
- "How many employees are in each department?" This needs `GROUP BY department` with `COUNT(*)` — not `SELECT DISTINCT department`, which only lists department names without counts.
- "Who has the third-highest salary?" That's `ORDER BY salary DESC OFFSET 2 LIMIT 1` — skip the top two, take the next one. Priya and Diego/Wren tie at similar salaries, which is exactly the kind of edge case interviewers plant on purpose.
- "Which departments have more than 2 employees?" You can't write `WHERE COUNT(*) > 2` — `COUNT(*)` doesn't exist yet at the row-filtering stage. That's a job for `HAVING COUNT(*) > 2`, applied after grouping.

Intuition

Before touching syntax, build the mental model an experienced engineer already has running in the background.

WHERE is a bouncer at a door. Rows show up one at a time, the bouncer checks a condition against that single row, and only matching rows get through. Nothing aggregate exists yet — no counts, no sums, no averages — because grouping hasn't happened. This is the single fact that explains almost every WHERE-related bug in this lesson.

NULL means "unknown," not "empty." SQL's three-valued logic (`TRUE` , `FALSE` , `UNKNOWN`) means comparing anything to NULL with `=` returns `UNKNOWN`, which behaves like `FALSE` when filtering. `NULL = NULL` is *also* unknown — even a NULL doesn't equal itself. The only way to test for NULL is with the special operators `IS NULL` / `IS NOT NULL`, which check for the state directly instead of comparing values.

DISTINCT and GROUP BY solve different problems. DISTINCT collapses duplicate rows into one — it's a dedup operation, nothing more. GROUP BY buckets rows so you can compute something *per bucket*: a count, a sum, an average. If the question asks "how many," "what's the total," or "what's the average, per X," you need GROUP BY. If it asks "what are the unique values," you need DISTINCT. Reaching for DISTINCT when the interviewer wants a per-department count is one of the most common tells that a candidate is pattern-matching rather than understanding.

HAVING exists because WHERE runs too early to see aggregates. The logical order of SQL execution is roughly: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY . By the time HAVING runs, groups already exist, so it can filter on COUNT , SUM , AVG , and other aggregates. WHERE runs before any of that — it only ever sees raw, ungrouped rows.

IN is a lookup, OR is a manual list. A chain of dept = 'Sales' OR dept = 'Engineering' OR dept = 'Marketing' and dept IN ('Sales', 'Engineering', 'Marketing') produce identical results, but IN is shorter, harder to get wrong with parentheses, and reads as a single intent rather than three fragile conditions stitched together.

Once these five mental models are solid, the actual SQL is almost mechanical.

Brute Force Solution

There isn't really a "brute force" version of writing a WHERE clause — but there is a brute-force *mental habit*, and it's worth naming because it's what most struggling candidates default to.

The habit: treat every filtering problem as "write conditions until the query runs," without first asking whether the condition needs IS NULL , whether the count belongs in WHERE or HAVING , or whether AND / OR need explicit parentheses.

- **Idea:** trial-and-error the syntax until the database stops complaining, then assume correctness.
- **Algorithm:** guess a clause, run it, eyeball the output, adjust if it looks wrong.
- **Advantage:** feels fast under interview pressure.
- **Disadvantage:** this is exactly how the NULL trap and the WHERE/HAVING mix-up survive undetected — both produce a *syntactically valid* query that silently returns the wrong rows. There's no error to catch you.
- **Time/space complexity:** not applicable — the cost here isn't computational, it's the cost of an undetected logic bug shipping to production.

This is the version that fails interviews and, worse, fails in real reporting pipelines where nobody double-checks the row count.

Optimal Solution

The optimal approach is a deliberate checklist applied *before* writing any WHERE clause, matched to the specific pattern the question is testing.

Pattern 1 — Missing data: always reach for `IS NULL` / `IS NOT NULL`, never `= NULL` or `!= NULL`.

Pattern 2 — Ambiguous logic: whenever AND and OR appear together, wrap the OR conditions in parentheses. SQL evaluates AND with higher precedence than OR, so `WHERE dept = 'Sales' AND salary > 70000 OR dept = 'Engineering'` doesn't mean what most people read it as — it silently becomes `(dept = 'Sales' AND salary > 70000) OR dept = 'Engineering'`, pulling in every Engineering row regardless of salary.

Pattern 3 — Unique values vs. per-group math:

Need	Use
List of distinct department names	<code>SELECT DISTINCT department</code>
Count of employees per department	<code>SELECT department, COUNT(*) ... GROUP BY department</code>

Pattern 4 — Nth-ranked row: sort, skip, then take one.

```
ORDER BY salary DESC
OFFSET 2
LIMIT 1
```

Pattern 5 — Filter timing: raw-row conditions go in WHERE; aggregate conditions go in HAVING.

```
FROM employees
WHERE department = 'Engineering'      -- filters raw rows first
GROUP BY department
HAVING COUNT(*) > 2                  -- filters the resulting groups
```

Pattern 6 — Multi-value match: replace OR chains with IN.

```
WHERE department IN ('Sales', 'Engineering', 'Marketing')
```

Python Code

Since this lesson is about query logic rather than a general-purpose algorithm, the "production code" here is the query itself, wrapped in a small Python utility that executes it safely against a database connection — useful for anyone automating these checks rather than running them by hand each time.

```
import sqlite3
from typing import Any

def run_query(connection: sqlite3.Connection, query: str, params: tuple = ()) -> list[dict[str, Any]]:
    """Execute a parameterized query and return rows as dictionaries."""
    connection.row_factory = sqlite3.Row
    cursor = connection.execute(query, params)
    return [dict(row) for row in cursor.fetchall()]

# Pattern 1: correct NULL check
MISSING_EMAIL_QUERY = """
    SELECT id, name
    FROM employees
    WHERE email IS NULL
    """

# Pattern 3: aggregate per group, not just distinct values
HEADCOUNT_BY_DEPT_QUERY = """
    SELECT department, COUNT(*) AS headcount
    FROM employees
    GROUP BY department
    """

# Pattern 4: third-highest salary
THIRD_HIGHEST_SALARY_QUERY = """
    SELECT name, salary
    FROM employees
    ORDER BY salary DESC
    LIMIT 1 OFFSET 2
    """

# Pattern 5: filter raw rows, then filter groups
LARGE_ENGINEERING_TEAMS_QUERY = """
    SELECT department, COUNT(*) AS headcount
    FROM employees
    WHERE department = 'Engineering'
    GROUP BY department
    HAVING COUNT(*) > 2
    """

# Pattern 6: IN instead of chained OR
TARGET_DEPARTMENTS_QUERY = """
    SELECT name, department
    FROM employees
    """
```

```
WHERE department IN ('Sales', 'Engineering', 'Marketing')
"""
```

Code Walkthrough

- `run_query` centralizes execution so every query above goes through the same parameterized path — this also means none of these are vulnerable to SQL injection, since real inputs would be passed via `params`, not string-formatted into the query.
- `MISSING_EMAIL_QUERY` demonstrates the fix for the NULL trap directly: `IS NULL`, not `= NULL`.
- `HEADCOUNT_BY_DEPT_QUERY` shows `GROUP BY` answering a "how many per bucket" question — notice `COUNT(*)` sits in the `SELECT` list, which only works because `GROUP BY` has already formed the buckets.
- `THIRD_HIGHEST_SALARY_QUERY` uses `LIMIT 1 OFFSET 2` — offset 2 skips the top two rows, then limit 1 grabs the next one, landing exactly on third place.
- `LARGE_ENGINEERING_TEAMS_QUERY` filters raw rows first with `WHERE` (only Engineering rows enter the group), then filters the resulting group with `HAVING` (only groups larger than 2). Doing this in the wrong order, or in the wrong clause, either throws an error or silently returns nothing.
- `TARGET_DEPARTMENTS_QUERY` replaces what would otherwise be a three-part `OR` chain with one `IN` clause — same result, less room for a missing parenthesis to break the logic.

Dry Run

Take `LARGE_ENGINEERING_TEAMS_QUERY` against the five-row `employees` table above.

1. **FROM employees** — start with all 5 rows.
2. **WHERE department = 'Engineering'** — keep only raw rows where department is Engineering: Priya, Diego, Wren survive; Asha and Marco are dropped.
3. **GROUP BY department** — the 3 surviving rows collapse into a single group: `Engineering`.
4. **HAVING COUNT(*) > 2** — the Engineering group has 3 rows, `3 > 2` is true, so the group survives.
5. **SELECT department, COUNT(*)** — output: `Engineering, 3`.

If you'd tried `WHERE COUNT(*) > 2` instead of `HAVING`, the query fails outright — `COUNT(*)` isn't valid until grouping has happened. If you'd used `department = NULL` anywhere in that chain by mistake, rows would vanish with no error at all — which is the more dangerous failure mode of the two.

Complexity Analysis

These are logical correctness problems, not algorithmic ones, so "complexity" here refers to query cost rather than Big-O:

- **WHERE filtering:** typically $O(n)$ over the scanned rows, or $O(\log n)$ per lookup if an index exists on the filtered column.
- **GROUP BY / HAVING:** requires an aggregation pass over filtered rows, generally $O(n \log n)$ if the engine sorts to group, or $O(n)$ with hash-based grouping.
- **ORDER BY + LIMIT/OFFSET:** $O(n \log n)$ for the sort; OFFSET does not reduce this cost — the engine still computes the full sorted order and then discards the skipped rows, which matters at scale but not for the small examples here.

The complexity isn't the interview risk in these questions — correctness is. A candidate can write an $O(n)$ query that is asymptotically fine and still return the wrong 3 rows because of a NULL comparison or precedence bug.

Alternative Solutions

For the Top-N pattern, window functions are a reasonable alternative once you're past the basics round:

```
SELECT name, salary
FROM (
    SELECT name, salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 3
```

`DENSE_RANK()` handles ties correctly (two people tied for 2nd both get rank 2, and rank 3 goes to the next distinct salary), whereas `OFFSET / LIMIT` treats ties as arbitrary row order. For an interview at the basics level, `ORDER BY ... LIMIT ... OFFSET` is the expected answer — but flagging the tie-handling difference to your interviewer shows you understand the tradeoff, which is a strong signal on its own.

Edge Cases

- **Empty table:** every query above should return zero rows without erroring — worth explicitly stating in an interview rather than assuming it.

- **All values NULL in the filtered column:** `WHERE email IS NULL` returns every row; `WHERE email = NULL` still returns none, which is the trap restated at its most extreme.
- **Ties at the Top-N boundary:** if two employees share the "third place" salary, `LIMIT 1 OFFSET 2` arbitrarily picks one of them unless the ORDER BY has a tiebreaker (e.g., `ORDER BY salary DESC, name ASC`).
- **HAVING with no matching groups:** if no department has more than 2 employees, the query returns zero rows, not an error — don't assume a query "isn't working" just because the result set is empty.
- **Single-row IN list:** `WHERE department IN ('Sales')` works identically to `WHERE department = 'Sales'` — a good sanity check that IN is just OR generalized.

Common Mistakes

1. Writing `WHERE column = NULL` instead of `WHERE column IS NULL`.
2. Mixing AND and OR without parentheses, silently changing which rows match.
3. Using `SELECT DISTINCT` when the question actually wants a per-group count via `GROUP BY`.
4. Trying to filter on an aggregate (`COUNT` , `SUM` , `AVG`) inside WHERE instead of HAVING.
5. Forgetting a tiebreaker column in ORDER BY when the ranking needs to be deterministic.
6. Defaulting to a long OR chain instead of IN, then missing a parenthesis somewhere in the chain.
7. Assuming an empty result set means the query is broken, rather than checking whether the filter is simply too strict.

Interview Questions

- "Why does `WHERE email = NULL` return zero rows even when NULL emails exist?"
- "What's the difference between `DENSE_RANK()` , `RANK()` , and `ROW_NUMBER()` for Top-N queries?"
- "Can you rewrite this OR chain using IN, and does the result change?"
- "In what order does SQL logically evaluate WHERE, GROUP BY, HAVING, and ORDER BY?"
- "How would you find the 2nd-highest salary in each department, not just overall?"

Similar Problems

- **LeetCode 176 — Second Highest Salary:** directly tests the Top-N / NULL-when-missing pattern from this lesson.
- **LeetCode 184 — Department Highest Salary:** combines GROUP BY reasoning with a join, a natural next step after this lesson.

- **LeetCode 262 — Trips and Users:** heavy WHERE/HAVING and NULL-handling practice in a messier, real-world-shaped schema.
- **LeetCode 1934 — Confirmation Rate:** tests DISTINCT vs GROUP BY judgment under a LEFT JOIN.

Key Takeaways

The basics are not a warm-up — they're the gate. Four checks cover most of what fails candidates: use `IS NULL` for missing data, choose DISTINCT or GROUP BY deliberately, parenthesize AND/OR explicitly, and know whether a condition belongs in WHERE or HAVING. None of these mistakes throw an error. All of them silently return the wrong rows, which is precisely why they're dangerous in an interview and in production reporting alike.

Watch the Video

For the full walkthrough with live queries and the pre-interview checklist in action, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around one idea: the fundamentals are where careers actually get made or stalled, and they deserve more rigor than they usually get. Each lesson takes a concept that looks "too basic to matter" and shows exactly where it quietly breaks real interviews and real systems.

Call To Action

If this lesson saved you from a mistake you didn't know you were making, like the video on YouTube and subscribe for the rest of the SQL Interview Basics series. Subscribe to the Substack for the full written breakdowns with extra examples that don't make it into the video. Drop a comment with the SQL trap that's bitten you before — chances are someone else in the comments has hit the exact same one. And if you know someone prepping for a SQL round, share this with them before their interview, not after.

Quick Review

Why do most SQL interview candidates fail basics questions?

They rush past edge cases like NULLs and filter logic instead of proving they understand core mechanics first.

WHERE filters rows, but which clause filters after grouping?

HAVING — WHERE applies before aggregation, HAVING applies to grouped/aggregated results.

Why does `col != value` silently drop NULL rows?

NULL comparisons return UNKNOWN, not true/false, so those rows get excluded — use IS NOT NULL explicitly.

DISTINCT vs GROUP BY — when do they behave differently?

DISTINCT dedupes selected columns; GROUP BY dedupes plus enables per-group aggregates like COUNT or SUM.

How do you get the top-N rows per query cleanly?

ORDER BY the ranking column, then LIMIT N — avoids nested subqueries for simple top-N cases.

Why replace a long OR chain on one column with something else?

Use IN (...) instead — same logic, more readable, and signals you know cleaner filtering idioms.

sql Lesson 29: SQL Interview Questions — Basics (This lesson covers the fundamental SQL concepts frequently tested in technical interviews, focusing on data retrieval and filtering logic. Understanding these core mechanics is essential for translating business requirements into accurate analytical queries. You should master these foundational topics to ensure data integrity and query efficiency when performing day-to-day database operations.) — free Python cheat sheet